

Metodología de Sistemas II

Trabajo Práctico 2

Testing Práctico y Contratos

1. Implementacion del patron AAA

Para este punto tome una funcion logica real de nuestro sistema: el calculo de total de facturacion del modulo medico (el mismo que refactorizamos en el TP de Clean Code), que aplica tres reglas de negocio encadenadas: factor reducido para pacientes particulares, descuento por obra social y descuento por fidelidad segun turnos previos. La funcion bajo prueba es `calcularTotal(SolicitudFacturacion)`, que valida la entrada, calcula el subtotal de estudios, aplica el descuento de cobertura y finalmente el de fidelidad.

La suite sigue rigurosamente las tres etapas en cada prueba. En Arrange se construyen unicamente los datos indirectos y el objeto bajo prueba (SUT), sin ejecutar logica de negocio; en Act se realiza una sola llamada al metodo bajo prueba, que es la unica accion con efectos del test; y en Assert se verifica el estado observable resultante, con tolerancia numerica para comparaciones de punto flotante. Cada prueba es independiente y autosuficiente: no comparte estado con las demas, de modo que cualquier falla pueda rastrearse a una unica regla de negocio.

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
import java.util.Arrays;
import java.util.Collections;
import java.util.List;

public class FacturacionTest {

    @Test
    public void pacienteParticular_aplicaFactorReducido() {

        List<Double> estudios = Arrays.asList(1000.0, 2000.0);
        SolicitudFacturacion solicitud =
            new SolicitudFacturacion(estudios, true, false,
0);
        Facturacion facturacion = new Facturacion();

        double total = facturacion.calcularTotal(solicitud);

        assertEquals(1500.0, total, 0.001);
    }

    @Test
    public void pacienteConObraSocial_aplicaDescuentoDeCobertura()
{

        List<Double> estudios = Arrays.asList(1000.0, 2000.0);
```

```

        SolicitudFacturacion solicitud =
            new SolicitudFacturacion(estudios, false, true,
0);
        Facturacion facturacion = new Facturacion();

        double total = facturacion.calcularTotal(solicitud);

        assertEquals(2700.0, total, 0.001);
    }

    @Test
    public void
pacienteConCuatroTurnosPrevios_aplicaDescuentoFijo() {

        List<Double> estudios = Arrays.asList(1000.0);
        SolicitudFacturacion solicitud =
            new SolicitudFacturacion(estudios, false, false,
4);
        Facturacion facturacion = new Facturacion();

        double total = facturacion.calcularTotal(solicitud);

        assertEquals(900.0, total, 0.001);
    }

    @Test
    public void
solicitudConTodasLasReglas_encadenaLosTresDescuentos() {
        List<Double> estudios = Arrays.asList(1000.0, 2000.0,
500.0);
        SolicitudFacturacion solicitud =
            new SolicitudFacturacion(estudios, true, true, 4);
        Facturacion facturacion = new Facturacion();

        double total = facturacion.calcularTotal(solicitud);

        assertEquals(1475.0, total, 0.001);
    }

    @Test
    public void
pacienteConTresTurnosPrevios_noAplicaDescuentoFijo() {

        List<Double> estudios = Arrays.asList(1000.0);

```

```

        SolicitudFacturacion solicitud =
            new SolicitudFacturacion(estudios, false, false,
3);
        Facturacion facturacion = new Facturacion();

        double total = facturacion.calcularTotal(solicitud);

        assertEquals(1000.0, total, 0.001);
    }

    @Test
    public void solicitudSinEstudios_devuelveCero() {
        SolicitudFacturacion solicitud =
            new SolicitudFacturacion(Collections.emptyList(),
false, false, 0);
        Facturacion facturacion = new Facturacion();

        double total = facturacion.calcularTotal(solicitud);

        assertEquals(0.0, total, 0.001);
    }

    @Test
    public void
solicitudConListaNula_fallaRapidoConExcepcionControlada() {
        SolicitudFacturacion solicitud =
            new SolicitudFacturacion(null, false, false, 0);
        Facturacion facturacion = new Facturacion();

        Throwable error =
assertThrows(IllegalArgumentException.class,
            () -> facturacion.calcularTotal(solicitud));
        assertEquals("La lista de precios de estudios no puede ser
nula.",
            error.getMessage());
    }
}

```

En la ultima prueba la etapa Act queda envuelta dentro de `assertThrows`, porque en escenarios de error la ejecucion y la aseveracion inicial coinciden: el framework ejecuta la llamada y captura la excepcion, y el Assert propiamente dicho verifica despues el tipo y el mensaje del error capturado. El resto de las pruebas mantiene las tres etapas separadas y visibles, que es el objetivo del patron: que cualquier lector identifique sin esfuerzo que se prepara, que se ejecuta y que se espera.

2. Informe de dobles de prueba ante la API externa de pagos

Contexto de aislamiento: Nuestro sistema procesa cobros a través de una API de pagos externa. Desde la arquitectura, el componente bajo prueba (el servicio de checkout) no conversa directamente con el proveedor: depende de una abstracción, el puerto PaymentGateway, y es un adaptador concreto el que traduce esa interfaz al protocolo HTTP del proveedor. Esa decisión de inversión de dependencias es la que hace posible el aislamiento en el laboratorio: en las pruebas unitarias el adaptador real se reemplaza por un doble que implementa el mismo puerto y se inyecta por constructor. Sin ese límite arquitectónico, cualquier prueba arrastraría red, latencia, costo por transacción real y no determinismo, dejando de ser unitaria. Dentro de esa frontera, la elección entre Stub y Mock depende de la dirección y el propósito de la colaboración.

Cuando es necesario un Stub: Un Stub es indispensable cuando la colaboración es una consulta que provee datos indirectos al componente bajo prueba, y lo que la prueba necesita es simplemente que esos datos existan y sean controlados. En el módulo de pagos, esto ocurre por ejemplo cuando el checkout consulta al gateway el estado de una transacción previa o el límite disponible de la tarjeta para decidir si puede continuar: la prueba no está evaluando esa consulta en sí, sino lo que el SUT hace con la respuesta. El stub responde siempre un valor fijo y predecible (estado APROBADO, límite suficiente), haciendo que el flujo sea determinista. Su protocolo es responder; no le interesa cuantas veces ni con que argumentos lo llamaron. Verificar con stubs es verificar estado: dado este dato indirecto, el resultado final del SUT debe ser este. Si en lugar de un stub se intentara usar la API real, la prueba quedaría atada a la cuenta de sandbox del proveedor y podría fallar por razones ajenas al código evaluado.

Cuando es indispensable un Mock: Un Mock es obligatorio cuando el resultado observable de la prueba no es un estado final dentro del SUT, sino la interacción misma con el colaborador, es decir, cuando el efecto relevante ocurre del otro lado del puerto. En pagos esto es lo habitual en los comandos: cobrar, reembolsar o notificar una auditoría no devuelven un dato que el checkout guarde, sino que producen un efecto externo que solo puede constataarse auditando el protocolo. Un mock registra cada llamada, sus argumentos, su orden y su cardinalidad, y hace fallar la prueba si el contrato se rompe. Ejemplos concretos donde el mock es insustituible: verificar que ante un pedido válido se ejecutó exactamente un cargo con el importe correcto y la clave de idempotencia esperada (un cargo duplicado es un incidente financiero que ningún assert sobre el estado interno detectaría); verificar que ante una validación fallida no se intentó ningún cargo; o verificar que al fallar el despacho se invocó el reembolso en el orden correcto respecto del cargo original. En todos esos casos no hay estado interno del SUT que confirme el comportamiento: la única evidencia es el historial de interacciones, y por eso el doble debe auditarlas.

Sintesis segun la arquitectura de aislamiento: La regla que usamos en el equipo es directa: las consultas que entran al SUT como datos indirectos se resuelven con stubs; los comandos que salen del SUT hacia el colaborador y constituyen el comportamiento esperado se verifican con mocks. Ambos viven en el mismo lugar arquitectonico (reemplazan al adaptador real en el puerto PaymentGateway), pero cumplen roles opuestos: el stub controla la entrada indirecta, el mock observa la salida indirecta. Tambien cuidamos el exceso de mocks: mockear colaboraciones irrelevantes para la prueba sobre-especifica el test y lo vuelve fragil ante refactorizaciones internas, mientras que un stub de mas nunca rompe una prueba por protocolo. En el modulo de pagos conviven ambos: stubs para estado de transaccion y limites, mocks para cargo, reembolso y notificacion de auditoria.

Como ilustracion minima, los dos dobles artesanales del puerto serian:

```
// Stub: provee dato indirecto controlado, no audita llamadas
public class PaymentGatewayStub implements PaymentGateway {
    public PaymentStatus status(String txId) { return
PaymentStatus.APPROVED; }
    public double availableLimit(String cardId) { return 10000.0;
}
}

// Mock: audita el protocolo de interaccion del comando charge
public class PaymentGatewayMock implements PaymentGateway {
    private int charges = 0;
    private double lastAmount;
    public void charge(String txId, double amount) { charges++;
lastAmount = amount; }
    public void assertSingleCharge(double expectedAmount) {
        if (charges != 1 || lastAmount != expectedAmount) {
            throw new AssertionError("Protocolo de cargo
violado");
        }
    }
}
}
```

3. Analisis de cobertura

Segmento elegido. Tome del sistema la regla que calcula el copago de una atencion segun edad, cobertura y urgencia, que tiene una estructura condicional compleja con tres decisiones encadenadas:

```
public double calcularCopago(int edad, boolean tieneObraSocial,
boolean esUrgente) {
```

```

double copago;
if (edad < 18 || edad >= 65) {
    copago = 0;
} else if (tieneObraSocial && !esUrgente) {
    copago = 500;
} else {
    copago = 1000;
}
if (!tieneObraSocial && esUrgente) {
    copago = copago + 250;
}
return copago;
}

```

Por que Branch Coverage es mas riguroso que Statement Coverage:

La cobertura de sentencias solo responde a la pregunta "esta linea de codigo se ejecuto al menos una vez", mientras que la cobertura de ramas responde a "cada decision fue evaluada tanto en su salida verdadera como en su falsa". La diferencia es critica en estructuras como D3, que es un if sin else: una prueba que atravesase su cuerpo alcanza el 100 por ciento de sentencias del fragmento, pero jamas ejercita el camino en que la condicion es falsa, que es exactamente el camino donde viven los pacientes que no deberian recibir el recargo. Imaginemos un defecto en que D3 quedara siempre verdadero (por ejemplo, alguien escribe if (true) o invierte el operador): la prueba con recargo seguiria pasando y ninguna sentencia quedaria sin ejecutar, pero todo paciente con obra social y atencion no urgente estaria pagando 250 de mas en silencio. Solo un test que ejercite la salida falsa de D3 (un paciente con obra social y urgencia falsa, o con urgencia verdadera y obra social verdadera) expone ese defecto, y ese test es justamente el que la cobertura de ramas exige y la de sentencias no. En terminos generales: statement coverage garantiza que el codigo fue tocado; branch coverage garantiza que la logica fue ejercitada en ambas direcciones, que es donde se esconden los errores de condicion invertida, else omitidos y cortocircuitos.

Casos de prueba para 100 por ciento de cobertura de ramas:

El segmento tiene tres decisiones y por lo tanto seis salidas a cubrir (D1 verdadera y falsa, D2 verdadera y falsa, D3 verdadera y falsa). Con los siguientes cuatro casos se alcanzan todas:

- Niño de 10 años, con obra social, no urgente. Esperado: 0. Cubre D1 verdadera (corta por el primer if) y D3 falsa.
- Adulto de 30 años, con obra social, no urgente. Esperado: 500. Cubre D1 falsa y D2 verdadera.
- Adulto de 30 años, con obra social, urgente. Esperado: 1000. Cubre D2 falsa (la urgencia anula el descuento de cobertura) y D3 falsa en un perfil con

recargo potencial: es el caso que detectaría el defecto del recargo mal aplicado.

- Adulto de 30 años, sin obra social, urgente. Esperado: 1250 (1000 de base mas 250 de recargo). Cubre D3 verdadera y refuerza D1 falsa y D2 falsa.

Con estos cuatro casos cada salida de cada decisión queda ejercitada al menos una vez: D1 verdadera en el caso 1, D1 falsa en los casos 2 a 4, D2 verdadera en el caso 2, D2 falsa en los casos 3 y 4, D3 verdadera en el caso 4 y D3 falsa en los casos 1 a 3. Vale aclarar que una suite ingenua de tres casos (1, 2 y 4) ya lograría el 100 por ciento de sentencias, porque entre ellos tocan todas las líneas, pero dejaría sin cubrir la salida falsa de D2 en el perfil con obra social y urgencia, quedando en cinco de seis ramas: ese es exactamente el hueco que la métrica de ramas vuelve visible y la de sentencias oculta.

Limite honesto de la métrica:

Incluso con 100 por ciento de ramas quedan sin explorar las combinaciones internas de las condiciones compuestas: por cortocircuito, en D1 un caso con edad < 18 verdadero nunca evalúa edad >= 65, y en D2 un caso con tieneObraSocial falsa nunca evalúa !esUrgente. Para ese nivel de rigor existe la cobertura de condiciones o MC/DC, que exige un caso por cada condición elemental cambiando su resultado de forma independiente. La cobertura de ramas no es el techo, pero sí es un piso bastante más serio que el de sentencias, y por eso es la que exigimos como mínimo en el módulo de facturación y copagos antes de integrar código.